	AIR UNIVERSITY
	DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING
	EXPERIMENT NO 4

Lab Title: **Doubly and Circular Linked lists**

Student Name: **Muammad Waqas** **Reg. No:** **240756**

Objective: Implementation of Doubly and circular linked lists in C++ l

LAB ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Ability to Conduct Experiment					
Ability to assimilate the results					
Effective use of lab equipment and follows the lab safety rules					

Total Marks: _____ Obtained Marks: _____

LAB REPORT ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Data presentation					
Experimental results					
Conclusion					

Total Marks: _____ Obtained Marks: _____

Date: _____ Signature: _____

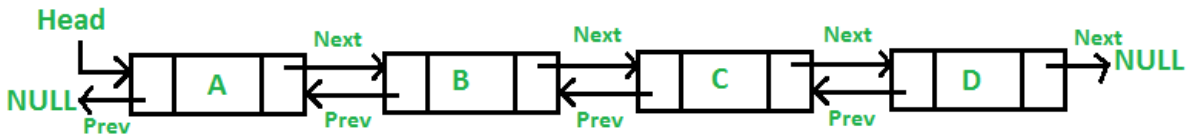
LABORATORY EXPERIMENT NO. 4

Doubly and circular linked lists

OBJECTIVE:

- To Enable efficient, bidirectional data management in C++ with O(1) insertion/deletion performance.
- Provide a structure that supports **circular, uninterrupted traversal** from any node back to itself.
- Implement a **dynamic data structure** superior to arrays for frequent modifications.

Doubly linked lists:



Circular Linked lists



Lab Tasks :

Sum-to-Tail

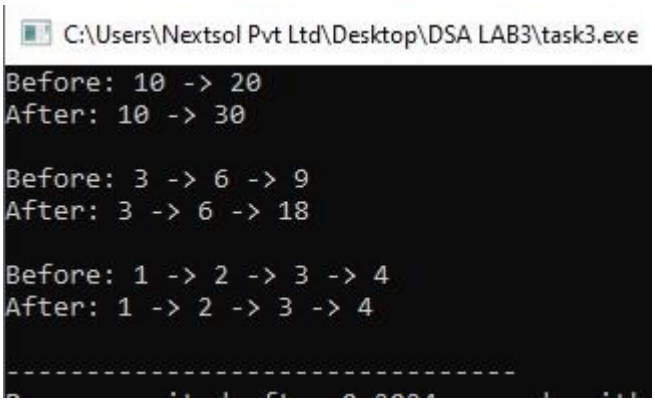
Task: If the list has exactly 2 or 3 nodes, compute the sum of all nodes and store in the last node's data (no new node). Otherwise do nothing.

Code:

```
#include<iostream>
using namespace std;
class Node{
public:
int data;
Node*next;
Node(int val){
data=val;
next=NULL;}};
class List{
private:
Node*head;
public:
List(){
head=NULL;}
void addNode(int val){
Node*newNode=new Node(val);
if(head==NULL)
head=newNode;
else{
Node*temp=head;
while(temp->next!=NULL)
temp=temp->next;
temp->next=newNode;}}
bool sumIfSize2or3(){
if(head==NULL)return false;
int count=0;
int sum=0;
Node*temp=head;
while(temp!=NULL){
sum+=temp->data;
count++;
temp=temp->next;}
if(count==2||count==3){
Node*tail=head;
while(tail->next!=NULL)
tail=tail->next;
tail->data=sum;
return true;}
return false;}
void display(){
Node*temp=head;
while(temp!=NULL){
cout<<temp->data;
if(temp->next!=NULL)
cout<<" -> ";
temp=temp->next;}
cout<<endl;}};
int main(){
List L;
L.addNode(10);
L.addNode(20);
cout<<"Before: ";
L.display();
L.sumIfSize2or3();
cout<<"After: ";
L.display();
List L2;
L2.addNode(3);
L2.addNode(6);
L2.addNode(9);
cout<<"\nBefore: ";
L2.display();
L2.sumIfSize2or3();
```

```
64    L2.sumIfSize2or3();
65    cout<<"After: ";
66    L2.display();
67    List L3;
68    L3.addNode(1);
69    L3.addNode(2);
70    L3.addNode(3);
71    L3.addNode(4);
72    cout<<"\nBefore: ";
73    L3.display();
74    L3.sumIfSize2or3();
75    cout<<"After: ";
76    L3.display();
77    return 0;
78  }
```

OUTPUT

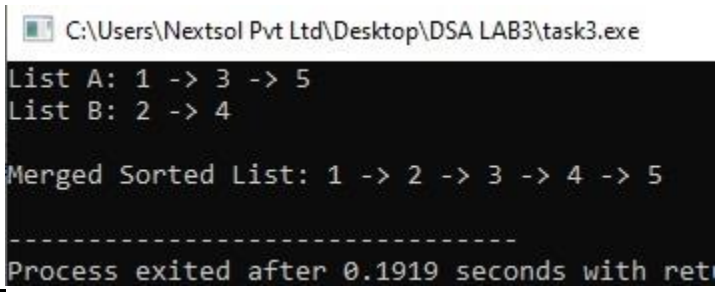


Task2:

Merge two sorted singly linked lists into one sorted list without creating new nodes (pointer splicing).

```cpp
#include<iostream>
using namespace std;
class Node{
public:
int data;
Node*next;
Node(int val){
data=val;
next=NULL;}};
Node*mergeSorted(Node*a,Node*b){
if(a==NULL)return b;
if(b==NULL)return a;
Node*head=NULL;
if(a->data<=b->data){
head=a;
a=a->next;
}else{
head=b;
b=b->next;}
Node*tail=head;
while(a!=NULL&&b!=NULL){
if(a->data<=b->data){
tail->next=a;
a=a->next;
}else{
tail->next=b;
b=b->next;}
tail=tail->next;}
if(a!=NULL)tail->next=a;
else tail->next=b;
return head;}
void printList(Node*head){
while(head!=NULL){
cout<<head->data;
if(head->next!=NULL)cout<<" -> ";
head=head->next;}
cout<<endl;}
void addNode(Node*&head,int val){
Node*newNode=new Node(val);
if(head==NULL)head=newNode;
else{
Node*temp=head;
while(temp->next!=NULL)
temp=temp->next;
temp->next=newNode;}}
int main(){
Node*a=NULL;
Node*b=NULL;
addNode(a,1);
addNode(a,3);
addNode(a,5);
addNode(b,2);
addNode(b,4);
cout<<"List A: ";
printList(a);
cout<<"List B: ";
printList(b);
Node*merged=mergeSorted(a,b);
cout<<"\nMerged Sorted List: ";
printList(merged);
return 0;}
```

OUPUT:


C:\Users\Nextsol Pvt Ltd\Desktop\DSA LAB3\task3.exe
List A: 1 -> 3 -> 5
List B: 2 -> 4
Merged Sorted List: 1 -> 2 -> 3 -> 4 -> 5
Process exited after 0.1919 seconds with ret

Conclusion:

- Merge and sort two arrays, and then use the resulting list's elements to efficiently perform data manipulation using the strengths of doubly and circular linked lists.
- Traverse the list forward from head to tail, calculating the cumulative sum of elements.
- Traverse the list backward (doubly list strength) to verify the sums or perform a reverse operation.
- Use the circular property to easily find the tail and append the final cumulative sum.
- Add the final cumulative sum as a new node at the end of the list.
- Maintain bidirectional links for easy node removal or insertion anywhere in the list.
- The final structure allows uninterrupted looping and easy access to both ends of the list for continuous data processing.